

Appunti pomeriggio BELLA 25-02-26

Nel corso della sessione pomeridiana si è affrontata la soluzione a un problema emerso in mattinata, relativo alla necessità di tracciare l'arrivo effettivo di un cliente, distinguendo così una prenotazione semplicemente programmata da una in cui il check-in è avvenuto. Il lavoro ha seguito un approccio bottom-up, partendo dal backend per risalire progressivamente verso il frontend.

Sul lato backend, il primo intervento ha riguardato l'entità **Booking**, alla quale è stato aggiunto un campo `private String status` per rappresentare lo stato della prenotazione. La scelta di usare una stringa è dettata dalla semplicità. Il **BookingDTO** è stato aggiornato di conseguenza, ma senza assegnare un valore predefinito al campo: questo perché al momento della creazione di una nuova prenotazione lo stato viene impostato direttamente nel modello come **"SCHEDULED"**. Successivamente, nel **BookingService**, è stato implementato il metodo `changeStatus`, che recupera la prenotazione tramite il suo ID, aggiorna lo stato con il valore ricevuto e salva le modifiche. Nel layer API è stato esposto un endpoint **PATCH** sul percorso `/ {id} / {status}`, che invoca il service e restituisce **HTTP 204** in caso di successo o **HTTP 404** se la prenotazione non viene trovata.

La seconda parte della sessione ha introdotto Git e il concetto di Source Control. Un repository Git è un archivio locale che conserva la storia completa di tutti i file di un progetto, rendendo possibile tornare a qualsiasi versione precedente. Lavorare su un progetto reale senza un sistema di versionamento significa rischiare di sovrascrivere o perdere irreversibilmente il lavoro proprio o altrui. Il comando `git init` trasforma una cartella comune in un repository, abilitando il tracciamento delle modifiche. Un file appena creato si trova nello stato **U** (untracked), ovvero Git ne è a conoscenza ma non lo monitora ancora. Con `git add` il file viene portato nello stato **A** (added), cioè aggiunto all'area di staging, pronta per essere inclusa nel prossimo commit. Il commit è l'operazione che salva definitivamente uno snapshot del progetto nel repository locale, accompagnato da un messaggio descrittivo. Solo dopo aver eseguito i commit in locale si procede a sincronizzare il lavoro con il repository remoto. Da notare che nei progetti Angular le librerie di dipendenze non vengono mai incluse nel push remoto perché troppo pesanti e riproducibili localmente tramite `npm install`.

Git, come già introdotto, è un sistema di versionamento locale: ogni repository creato con `git init` vive sulla macchina dello sviluppatore e conserva la storia dei commit in locale. GitHub è invece il corrispettivo remoto, ovvero un servizio cloud che ospita quel repository e lo rende accessibile e condivisibile. Il flusso di lavoro standard prevede di lavorare in locale, effettuare i commit per salvare le versioni e infine pubblicare il ramo sul repository remoto tramite "Publish Branch", scegliendo se renderlo pubblico o privato. La distinzione tra diff visivi in VS Code, dove i cambiamenti vengono evidenziati colorando le righe aggiunte o modificate, serve esattamente a tenere traccia di cosa è cambiato rispetto all'ultimo commit registrato.

Tornando all'applicazione, il passo successivo è stato integrare il campo `status` anche nel frontend. La prima modifica ha riguardato il modello TypeScript dell'interfaccia **Booking**, a cui è stata aggiunta la proprietà `status: string`. Questo è necessario affinché Angular sia consapevole dell'esistenza di quel campo quando riceve i dati dal backend.

In seguito è stato creato nel **BookingService** un metodo `doCheckIn`, il quale riceve l'ID della prenotazione e costruisce una chiamata HTTP di tipo **PATCH** verso l'endpoint `/id/executed`,

passando `null` come body poiché non è necessario inviare dati aggiuntivi. Il metodo restituisce un `Observable<Booking>` che il chiamante dovrà sottoscrivere.

Il componente `BookingRow`, che si occupa di visualizzare una singola riga prenotazione, ha bisogno di invocare questo servizio: viene quindi iniettato il `BookingService` tramite la funzione `inject`. All'interno del componente è stato definito il metodo `doCheckIn()`, che legge l'ID della prenotazione usando l'operatore `??` — detto operatore di coalescenza nulla — il quale restituisce il valore di sinistra se presente, oppure il valore di destra (in questo caso `0`) se il valore di sinistra è `null` o `undefined`. Una volta ottenuto l'ID, il metodo sottoscrive la chiamata al service: nel callback `next` aggiorna localmente il segnale `booking` usando `update`, sostituendo il campo `status` con il valore `"executed"` tramite spread operator, così da non dover ricaricare i dati dal server. Il callback `error` si limita a loggare l'errore in console.

Nel template HTML del componente è stata aggiunta la logica condizionale: con `@if(booking().status !== 'executed')` viene mostrato il pulsante "Check-in", che al click invoca `doCheckIn()`. Una volta che lo stato diventa `executed`, la condizione diventa falsa e il pulsante scompare, riflettendo visivamente il cambio di stato della prenotazione.

Il risultato finale è visibile nell'interfaccia della pagina "Arrivi di oggi": ogni riga mostra il pulsante "Check-in" solo per le prenotazioni ancora in stato `SCHEDULED`, e il pulsante sparisce dinamicamente non appena il check-in viene confermato, senza necessità di ricaricare la pagina.

Per l'evoluzione successiva della funzionalità, la logica dei pulsanti dovrà seguire queste regole: quando una prenotazione è in stato `SCHEDULED` devono essere visibili due pulsanti, uno per confermare il check-in e uno per annullare la prenotazione impostandola come `CANCELED`. Quando invece la prenotazione è in stato `EXECUTED`, il pulsante che deve comparire riguarda le pulizie della stanza, ma solo a partire dal giorno del check-out in poi: in quel momento diventa accessibile un campo booleano `cleaned`, inizialmente `false`, che l'operatore può portare a `true` per segnalare che la camera è stata pulita. I pulsanti visibili dipendono quindi da una combinazione tra lo stato della prenotazione e la data corrente rispetto alla data di check-out.